

RTL Obfuscation

本项目通过一致改写 SystemVerilog RTL 名称来加密源码，并可使用 `mapping.json` 恢复原文件。

输入后缀边界：独立编译单元支持小写 `.sv`、`.v`，被 include 的物理头文件支持小写 `.svh`、`.vh`。

四种后缀都沿用当前 PySlang 的 SystemVerilog 语义模式解析；这不是一套按后缀切换的 legacy Verilog parser。大写 `.V`、`.VH`、`.txt` 和把 `.vh` 当作单文件 source unit 会稳定失败。

3 分钟快速开始

前提：在仓库根目录使用 Python 3.10 或更高版本，并已安装 PySlang 11.x；如未安装，跳到 [安装](#安装)。

复制下面命令即可完成一次加密和恢复：

```
quick_work="$(mktemp -d /tmp/rtl-obfuscation-quick.XXXXXX)"
python rtl_encrypt.py \
  --input 11_supported_obfuscation.sv \
  --source-root rtl_samples \
  --output-dir "$quick_work/gate"
cat "$quick_work/gate/encryption_summary.txt"
python rtl_decrypt.py \
  --map "$quick_work/gate/mapping.json" \
  --gate-dir "$quick_work/gate" \
  --output-dir "$quick_work/restored"
cmp rtl_samples/11_supported_obfuscation.sv \
  "$quick_work/restored/11_supported_obfuscation.sv"
```

如何判断成功：

- 加密命令退出码为 `0`，终端 JSON 中 `action\_counts.rename` 大于 `0`；
- `summary.strict\_compile\_passed` 和 `summary.restored\_byte\_identical` 均为 `true`；
- 最后一条 `cmp` 没有输出，表示公开恢复结果与原文件逐字节一致。

输出目录只在全部检查成功后发布。失败时首行是稳定错误码，第二行会给出检查建议，例如：

```
error: CLI_VNEXT_INPUT_INVALID
hint: 请检查输入模式和路径；project-root 模式必须同时提供 --source-root 与 --top。
```

用在自己的工程

真实工程优先使用显式 filelist。它能固定编译顺序，并配合 `--include-dir`、`--define` 准确提供编译环境。第一次建议只选少量类型，例如：

```
python rtl_encrypt.py \
  --filelist design.f \
  --source-root <项目源码根目录> \
  --top <顶层模块名> \
  --category signals \
  --category instances \
  --output-dir <尚不存在的输出目录>
```

确认严格编译、实际改名数和恢复结果后，再用新的输出目录逐类增加 `--category`。不提供 `--category` 时会使用当前模式的默认范围，但这不表示任意工程的全部默认类型都已稳定支持。可选值和保守边界见 [SystemVerilog 可加密类型表](docs/systemverilog_renaming_table.md)。

看懂结果

加密命令的 JSON 和 `encryption\_summary.txt` 会直接报告：

- `rename` : 实际改名的对象 ;
- `preserve` : 因边界或策略保持原名的对象 ;
- `unsupported` : 当前证据不足、为避免错误而不改名的对象 ;
- `modified\_tokens` : 实际改写的源码 token 数。

`rename=0` 表示本次没有发生有效加密，不能把它理解为所选类型已经完整支持。详细记录位于：

```
<output-dir>/mapping.json
<output-dir>/metrics.json
<output-dir>/mapping_table.csv
<output-dir>/encryption_summary.txt
```

`mapping.json` 同时用于恢复；`mapping\_table.csv` 只列出实际替换项。严格编译和逐字节恢复是发布 gate 的必要条件，但复杂工程仍应运行自身仿真、综合或 [Formal 验证流程](docs/formal_verification.md)。

三种加密模式

| 模式 | 必要输入 | 适用方式 |

| --- | --- | --- |

| 单文件 | `--input`、`--source-root` | 快速试用或独立`.sv/.v`文件；只处理内部名称 |

| filelist | `--filelist`、`--source-root`，`--top` 可选 | 真实工程首选；按 filelist 编译顺序处理全部文件 |

| project-root | `--source-root`、`--top` | 从 top 自动发现源码；适合目录和依赖都完整的工程 |

单文件：

```
python rtl_encrypt.py \
--input <input_file.sv_or_v> \
--source-root <源码根目录> \
--output-dir <加密输出目录>
```

Filelist：

```
python rtl_encrypt.py \
--filelist design.f \
--source-root <源码根目录> \
--top <可选的顶层模块名> \
--output-dir <加密输出目录>
```

仓库示例使用`rtl\_samples/example\_fifo/design.f`、源码根目录

`rtl\_samples/example\_fifo` 和 top `fifo\_top`。不提供`--top`时只处理 module 内部名称；

提供后会一致处理该 top 使用的跨 module 名称，同时保留 top module 名称和对外端口。

filelist 中的`.sv/.v`是 source unit；`.svh/.vh`只作为 include 物理文件进入 gate、mapping

和恢复清单，不会被写入 canonical `design.f`。

Project-root：

```
python rtl_encrypt.py \
--source-root <项目源码根目录> \
--top <顶层模块名> \
--output-dir <加密输出目录>
```

Project-root 不使用`--input`或`--filelist`。它是方便入口；若自动发现受宏、include 或文件顺序

影响，请改用显式 filelist。当前版本还会保留 top module 内部直接声明的 interface 实例名。

解密

使用加密时生成的`mapping.json`和对应 gate，无需提供原始源码：

```
python rtl_decrypt.py \
--map <加密输出目录>/mapping.json \
--gate-dir <加密输出目录> \
--output-dir <恢复输出目录>
```

恢复目录运行前必须不存在。需要额外保存恢复报告时增加 `--report <恢复报告.json>`。

常用参数

选项	用法
`--include-dir PATH`	添加 include 目录，可重复使用；相对路径以源码根目录为基准
`--define NAME[=VALUE]`	添加预处理宏，可重复使用，例如 `--define SYNTHESIS`
`--category NAME`	只处理指定类型，可重复使用
`--encryption-rate RATE`	目标加密率，范围为 `0 < RATE <= 1`；不是标识符数量的精确比例
`--name-length N`	新名称长度，最小为 `4`，默认值为 `20`
`--map PATH`	自定义映射报告路径；默认是 `/mapping.json`
`--metrics PATH`	自定义覆盖率报告路径；默认是 `/metrics.json`
`--report PATH`	解密时额外保存恢复报告

查看全部参数和三种输入模式提示：

```
python rtl_encrypt.py --help
python rtl_decrypt.py --help
```

安装

仓库提供 CPython 3.11、Linux x86_64、glibc 2.17 或更高版本使用的 PySlang 11.0.0 wheel。

推荐在虚拟环境中安装：

```
python --version
python -m venv .venv
source .venv/bin/activate
python -m pip install --no-index --no-deps \
    wheel/pyslang-11.0.0-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.whl
python -c "import pyslang; print(pyslang.__version__)"
```

其他环境请准备匹配的 PySlang 11.x，详见

[PySlang 源码编译与离线部署指南](docs/pyslang源码编译与离线部署指南.md)。

更多文档

- [SystemVerilog 可加密类型表](docs/systemverilog_renaming_table.md)：类型选择和保守边界；
- [Formal 验证流程](docs/formal_verification.md)：独立检查功能等价性；
- [开发文档索引](docs/development/README.md)：只面向维护者，不是首次使用必读内容。